

QLPP

1. Introduzione

QLPP (Quantum Logic Programming in Prolog) è un framework dichiarativo realizzato in Prolog per la progettazione automatica di circuiti quantistici a partire da espressioni in logica booleana.

Il framework consiste dei seguenti componenti:

- Definizione degli operatori
- Astrazione dell'hardware e gestione dello stato
- Compilazione
- Operatori speciali, estensioni e utility

2. Definizione degli operatori

La definizione degli operatori include le seguenti clausole:

- **:- op**: direttive di configurazione degli operatori logici per l'interprete Prolog, in particolare il tipo (infisso/prefisso), l'associatività e un valore di precedenza (più basso il valore, più alta la precedenza);
- **q_operator**: controllo di validità degli operatori;
- **gate**: mapping di ciascun operatore nella porta quantistica (gate) corrispondente, con specifica del numero di qbits richiesti.

3. Astrazione dell'hardware e gestione dello stato

L'astrazione dell'hardware e la gestione dello stato includono le seguenti clausole:

- **:- asserta(qnn(0)) / :- asserta(cnn(0))**: direttive di inizializzazione del contatore di qbits/cbits per l'interprete Prolog;
- **qbit_number / cbit_number**: incremento del contatore di qbits/cbits e ritorno dell'indice del qbit/cbit aggiunto;
- **qvalue / cvalue**: come sopra, ma con controllo dell'input e ritorno di un valore default di fail nel caso che non sia una variabile libera;
- **reset_qbit_number / reset_cbit_number**: reset del contatore di qbits/cbits;
- **make_qbit / make_cbits**: istanza del registro di qbits/cbits, con specifica in input della sua dimensione ed inizializzazione con valori default di identità;
- **fill_register**: inserimento delle espressioni nel registro e di valori invariati ove non specificate;
- **filter**: semplificazione post compilazione, con rimozione del primo segmento circuitale fittizio (con valori default di identità) e di wrapper ridondanti.

4. Compilazione

La compilazione include due fasi principali: sintesi logica e layering.

Viene avviata globalmente dalle seguenti clausole:

- **qcircuit**: chiamata di **circuit_tree** e di **circuit_tree_log** (attivabile con “debug(circuit_tree).”) per la sintesi logica e di **layers** per il layering;
- **transform**: chiamata del passo precedente per ciascun qbit target del registro e ritorno del segmento circuitale;
- **build_circuit** con identificativo **exp(Exp)**: ricezione in input delle espressioni e chiamata di **fill_register**, **transform** e **filter**;
- **complete_circuit** con identificativo **exp(Exp)**: come sopra, ma con alla fine l’append del segmento circuitale ad un circuito preesistente.

4.1. Sintesi logica

La sintesi logica include le seguenti clausole:

- **normalize**: pattern matching di una espressione, con scomposizione e validazione secondo la struttura delle operazioni unarie e binarie;
- **translate**: traduzione di una espressione in albero sintattico (liste annidate), che stabilisce le dipendenze logiche per un qbit target del registro, con mapping di ciascun operatore al gate corrispondente;
- **circuit_tree**: chiamata dei due passi precedenti;
- **circuit_tree_log**: stampa la traduzione se non banale.

4.2. Layering

Il layering include le seguenti clausole:

- **level_gates**: estrazione di ogni gate del livello corrente dell’albero sintattico;
- **subtrees**: estrazione di ogni sottoalbero del livello corrente dell’albero sintattico per l’elaborazione del livello successivo;
- **layers**: chiamata dei due passi precedenti e trasformazione dell’albero sintattico in layers (lista di liste), che stabiliscono lo scheduling di esecuzione circuitale per un qbit target del registro.

5. Operatori speciali, estensioni e utility

5.1. Operatori speciali

Gli operatori speciali (quantistici come SWAP e Fredkin, o di misurazione) sono gestiti dalle seguenti clausole:

- **make_exps**: mapping dell’operatore speciale alle corrispondenti espressioni tradotte per ciascun qbit/cbit target (la fase di compilazione standard viene saltata);
- **measured_exp** / **measure_exp**: mapping specifici, a supporto del passo precedente, per la misurazione su una lista arbitraria di qbits/cbits;
- **apply_operator**: chiamata di **make_exps** e **fill_register**;
- **build_circuit** con identificativi **operator(QOP)** e **measurement**: ricezione in input rispettivamente di un operatore quantistico speciale o dell’operatore di misurazione, e chiamata del passo precedente;

- **complete_circuit** con identificativi **operator(QOP)** e **measurement**: come sopra, ma con alla fine l'append del segmento circuitale ad un circuito preesistente.

5.2. Estensioni

Le estensioni sono operatori complessi strutturati su più segmenti circuitali e sono gestiti in **build_circuit** e **complete_circuit** con i seguenti identificativi:

- **n-tof**: costruzione di Toffoli in sequenza per un numero arbitrario di qbits, con gestione delle ancille necessarie ai risultati intermedi e due possibili modalità di utilizzo:
 - o **conj** (default): costruzione dell'operatore di coniugazione, ovvero della sequenza compute-action-uncompute;
 - o **chain**: costruzione della catena di Toffoli senza uncompute;
- **operator(qOR)**: operatore qOR trasformato con legge di De Morgan e scomposto in negazione dei controlli, azione sul target (con Toffoli), negazione del target e uncompute dei controlli;
- **n-or**: catena di qOR trasformata combinando i due passi precedenti, con scomposizione in negazione dei controlli (con **negate**), azione sul target (con **n-tof**), negazione del target e uncompute dei controlli;
- **parity**: parity check di un numero arbitrario di qbits su un'ancilla (con **qXOR_n1**), memorizzazione del risultato su un target e uncompute;
- **ghz**: entanglement di un numero arbitrario di qbits (con **qXOR_1n**),
- **incr** / **decr**: incremento/decremento di un numero arbitrario di qbits (con **n-tof**).

5.3. Utility

Le utility includono le seguenti clausole:

- **superpose**: superposizione (con qH) su una lista arbitraria di qbits;
- **negate**: negazione (con qNOT) su una lista arbitraria di qbits;
- **bw_qXOR**: bitwise qXOR (in-place) su due liste arbitrarie di qbits;
- **qXOR_1n**: qXOR "uno a molti" (1 qbit di controllo, n qbit target);
- **qXOR_n1**: qXOR "molti a uno" (n qbit di controllo, 1 qbit target);
- **qrange** / **crange**: estrazione di un intervallo di qbits/cbits;
- **qidx_check** / **cidx_check**: verifica esistenza qbits/cbits;
- **compress**: compressione dell'output con la rimozione dei valori identità;
- **portray**: formattazione più leggibile per la stampa degli operatori logici.

6. Caso d'uso: parity check

6.1. Query Prolog

```
% parity check query
parity_query(Circuit) :-
    % reset contatori
    reset_qbit_number, reset_cbit_number,

    % inizializzazione dei registri
    make_qbits(6,QRin), make_cbits(1,CRin),

    % lista qbits di controllo
    qrange(4,CQbits),

    % lista qbits ancilla e target
    qrange(4,6,ATQbits),

    % compilazione del parity check
    build_circuit(QRin,parity,CQbits,ATQbits,QRout),

    % operatore speciale di misurazione
    complete_circuit(QRin,CRin,QRout,measurement,[q(5),c(0)],Circuit).
```

6.2. Sintesi logica

Parity compute: formula $\rightarrow$ tree

```
Target: q(4)
Formula: q(3) qXOR q(2) qXOR q(1) qXOR q(0) qXOR q(4)
Tree: [cnot(q(3),q(4)),[q(3)],[cnot(q(2),q(4)),[q(2)],[cnot(q(1),q(4)),[q(1)],[cnot(q(0),q(4)),[q(0)],[q(4)]]]]]
```

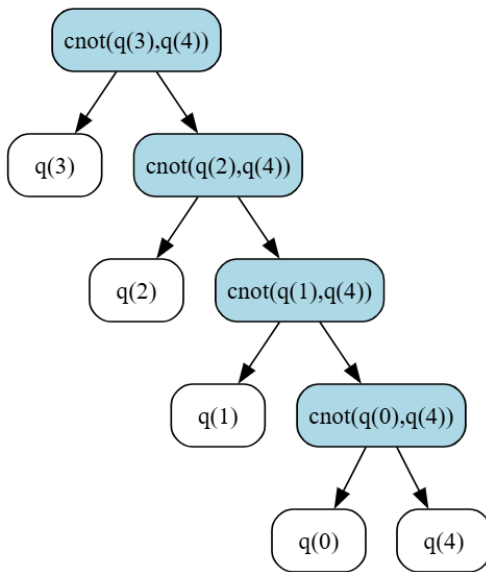
Parity target action: formula $\rightarrow$ tree

```
Target: q(5)
Formula: q(4) qXOR q(5)
Tree: [cnot(q(4),q(5)),[q(4)],[q(5)]]
```

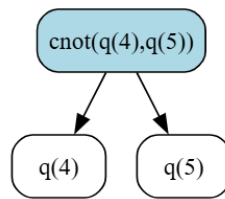
Parity uncompute: formula $\rightarrow$ tree

```
Target: q(4)
Formula: q(0) qXOR q(1) qXOR q(2) qXOR q(3) qXOR q(4)
Tree: [cnot(q(0),q(4)),[q(0)],[cnot(q(1),q(4)),[q(1)],[cnot(q(2),q(4)),[q(2)],[cnot(q(3),q(4)),[q(3)],[q(4)]]]]]
```

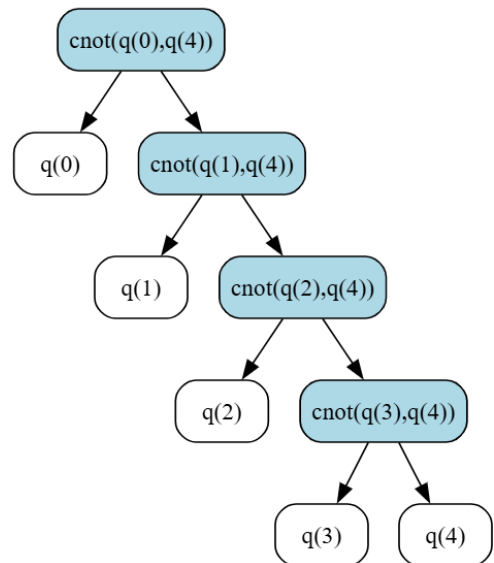
Parity compute



Parity target action



Parity uncompute



6.3. Layering

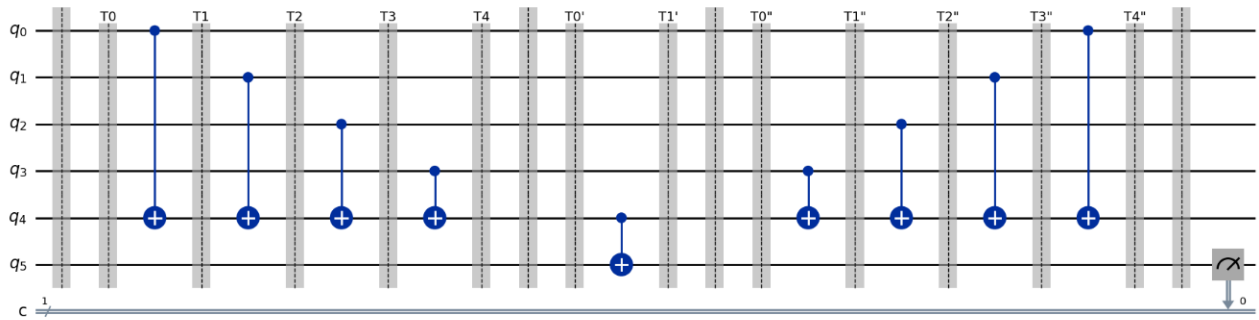
Circuito Prolog

```

% Circuit = [[q(0):q(0),q(1):q(1),q(2):q(2),q(3):q(3),
%           q(4):[[cnot(q(0),q(4))],
%               [cnot(q(1),q(4))],
%               [cnot(q(2),q(4))],
%               [cnot(q(3),q(4))]],
%           q(5):q(5)],
%           [q(0):q(0),q(1):q(1),q(2):q(2),q(3):q(3),q(4):q(4),
%           q(5):[[cnot(q(4),q(5))]]],
%           [q(0):q(0),q(1):q(1),q(2):q(2),q(3):q(3),
%           q(4):[[cnot(q(3),q(4))],
%               [cnot(q(2),q(4))],
%               [cnot(q(1),q(4))],
%               [cnot(q(0),q(4))]],
%           q(5):q(5)],
%           [q(0):q(0),q(1):q(1),q(2):q(2),q(3):q(3),q(4):q(4),
%           q(5):[measured(c(0),q(5))],
%           c(0):[measure(q(5),c(0))]]]

```

Circuito Qiskit



Le barriere etichettate dividono i layer temporali (ciò che è stato definito 'layering').

Le barriere non etichettate dividono i segmenti (o layer) circuitali.

I qbits di controllo ($q(0)$, $q(1)$, $q(2)$, $q(3)$) possono avere stato iniziale qualsiasi,
il qbit ancilla ($q(4)$) e il qbit target ($q(5)$) devono avere stato iniziale $|0\rangle$.